

Blokk 1

Sources og seeds

Hva er en source i dbt?

En **source** er en tabell som dbt ikke eier — den er lastet inn av et annet system.

Ved å registrere kilder i dbt får du:

- Mulighet til å referere til dem med `{{ source() }}` i stedet for hardkodede tabellnavn
- Automatisk linjeavstamning fra kilde til mart
- Freshness-sjekker som varsler når data er for gammelt

```
# models/sources.yml (filnavn. Et hvilket som helst gyldig finavn er ok)
sources:
  - name: crm # <- Navn på kilden
    schema: raw # <- Schema for kilden. Valgfritt; hvis ikke oppgitt brukes name
  tables:
    - name: crm_customers # <- navnet på tabellen i kilden
    - name: crm_contracts
```

`{{ source() }}` i en modell

I stedet for å skrive `raw.crm_customers` direkte i SQL bruker du:

```
select *  
from {{ source('crm', 'customers') }}
```

dbt peker til riktig database og schema basert på YAML-konfigurasjonen, slik at hvis schema-navnet endrer seg, endrer du det ett sted.

Freshness

dbt kan sjekke hvor ferske kildedataene er ved å lese en tidsserie-kolonne.

```
sources:
  - name: crm
    schema: raw
    loaded_at_field: _loaded_at # <- Hvilken kolonne som brukes til å angi rekkefølge
    freshness:
      warn_after: {count: 24, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: crm_customers
      - name: crm_contracts
      freshness:
        warn_after: {count: 6, period: hour} # override per tabell
```

Kjøres med:

```
dbt source freshness
```

Freshness — hva betyr terskelverdiene?

`warn_after` og `error_after` definerer hvor gammelt det nyeste innlastede datapunktet kan være.

- **warn**: dbt rapporterer en advarsel, jobben fortsetter
- **error**: dbt feiler — nyttig i en pipeline der downstream-kjøring ikke gir mening på foreldet data

Til diskusjon

- Hva er et realistisk freshness-SLA for hvert kildesystem?
- Bør CRM og billing ha samme terskel?
- Hva skjer i helger og helligdager?

Hva er et seed?

Et **seed** er en CSV-fil som dbt laster inn som en tabell i databasen.

Passer for små, stabile referansedata som ikke finnes i noe kildesystem — for eksempel en mapping-tabell eller en produktkatalog.

```
dbt-project/  
└─ seeds/  
   └─ catalog_products.csv
```

Lastes inn med:

```
dbt seed
```

Deretter kan andre modeller referere til den med `{{ ref('catalog_products') }}`.

Seeds i YAML

Seeds kan beskrives og konfigureres i YAML på samme måte som modeller.

```
# seeds/_seeds.yml
seeds:
  - name: catalog_products
    description: >
      Produktkatalog med alle abonnementstyper selskapet tilbyr.
      Inneholder historiske produktversjoner med valid_from / valid_to.
    columns:
      - name: product_code
        description: Unik produktkode. Kan gjenbrukes på tvers av versjoner.
      - name: valid_to
        description: Null for gjeldende produkter. Satt for utgåtte versjoner.
```

Du kan også overstyre kolonnetyper direkte i YAML:

```
config:
  column_types:
    monthly_fee_eur: float64
    valid_from: date
```


Kilder

Vi har data fra fire kildesystemer, alle i BigQuery-datasettet `raw`:

SYSTEM	TABELLER
CRM	<code>crm_customers</code> , <code>crm_contracts</code>
Fakturering	<code>billing_invoices</code> , <code>billing_payments</code>
Nettverk	<code>network_service_activations</code> , <code>network_incidents</code>
Produktkatalog	<code>catalog_products</code>

I dette blokket registrerer vi kildene og gjør oss kjent med dataene.

Blokk 2

Staging-modeller og materialisering

Hva er en staging-modell?

En staging-modell er en **1-til-1-representasjon av én kildetabell** — rensset, men ikke tolket.

Det er det eneste stedet i prosjektet der vi:

- Gir kolonner konsistente navn
- Caster datatyper til riktige typer
- Løser opp kildesystemets tekniske særegenheter

Staging-modeller inneholder **ingen forretningslogikk** og **ingen joins**.

```
stg_crm__customers.sql    ← én modell per kildetabell
stg_crm__contracts.sql
stg_billing__invoices.sql
stg_billing__payments.sql
```

Hva hører hjemme i staging?

HØRER HJEMME	HØRER IKKE HJEMME
Kolonnenavn → standard navn	Joins mellom tabeller
<code>VARCHAR</code> → <code>DATE</code> , <code>NUMERIC</code>	Beregninger på tvers av rader
JSON-utpakking (enkle, flate felt)	Forretningsregler og flagg
Filtrering av tekniske metadata-rader	Aggregering
<code>COALESCE</code> for enkle standardverdier	Historisering / SCD-logikk

Navngiving i staging

Standardiser kolonnenavn på tvers av alle staging-modeller:

```
select
  -- rename: fjern kildesystemets særegenheter
  custID           as customer_id,
  full_name        as customer_name,

  -- cast: gi kolonnen riktig type
  cast(created_at as date)           as created_date,
  safe_cast(monthly_fee as numeric)  as monthly_fee_eur,
  parse_date('%Y%m%d', invoice_date) as invoice_date,

  -- beholdes som de er
  email,
  phone,
  status

from {{ source('crm', 'crm_customers') }}
```

`safe_cast` i BigQuery returnerer `null` i stedet for å feile ved ugyldige verdier — nyttig under utforsking.

Hva er materialisering?

Materialisering bestemmer **hvordan dbt skriver modellen til databasen**.

TYPE	HVA DBT GJØR	TYPISK BRUK
<code>view</code>	Oppretter en view — ingen data lagres	Staging, enkle transformasjoner
<code>table</code>	Materialiserer hele resultatet som en tabell	Tunge intermediate-modeller
<code>incremental</code>	Legger til / oppdaterer kun nye rader	Store faktatabeller, events
<code>ephemeral</code>	Kompileres inn som en CTE — finnes ikke i DB	Gjenbrukbare mellomsteg

Staging-modeller er nesten alltid `view` — de er tynne og brukes som grunnlag for neste lag, ikke som sluttpunkt.

Materialisering i YAML-konfigurasjon

Materialisering settes i YAML — ikke i SQL-filen.

```
# models/staging/_staging.yml
models:
  - name: stg_crm__customers
    config:
      materialized: view
```

Eller på mappenivå i `dbt_project.yml`:

```
models:
  mitt_prosjekt:
    staging:
      +materialized: view      # gjelder alle modeller i staging/
    intermediate:
      +materialized: table
    marts:
      +materialized: table
```

+ foran en nøkkel betyr at den arves nedover i mappestrukturen.

View vs. table

En `view` beregnes på nytt hver gang den spørres. Det er greit når:

- Modellen er rask å kjøre
- Den brukes sjelden eller av andre modeller, ikke sluttbrukere
- Du vil alltid se ferske data

Bytt til `table` når:

- Modellen tar lang tid å kjøre og refereres ofte
- Downstream-modeller bygger på den og du vil ikke gjøre jobben om igjen
- Sluttbrukere spør direkte mot modellen

Staging er nesten alltid `view`. Intermediate og mart er ofte `table`. Start med `view` og bytt når du har grunn til det.

Datasett: staging

Vi skal bygge én staging-modell per kildetabell:

STAGING-MODELL	KILDE	HVA SOM MÅ FIKSES
stg_crm__customers	crm_customers	custID → rename, JSON-felt, created_at cast
stg_crm__contracts	crm_contracts	monthly_fee VARCHAR → NUMERIC
stg_billing__invoices	billing_invoices	InvoiceID → rename, YYYYMMDD → DATE, amount_eur → NUMERIC
stg_billing__payments	billing_payments	Ryddig — minimalt å gjøre
stg_network__service_activations	network_service_activations	JSON device_info — diskusjon: her eller intermediate?
stg_network__incidents	network_incidents	customer_ref — behold navn eller rename?
stg_catalog__products	catalog_products	Ryddig — seed, bruker ref() ikke source()

Blokk 3

Intermediate-modeller

Hva er et intermediate-lag?

Intermediate-modeller kombinerer og beriker staging-modeller. De forbereder entiteter for konsumpsjon — uten å ta stilling til hvem som konsumerer dem.

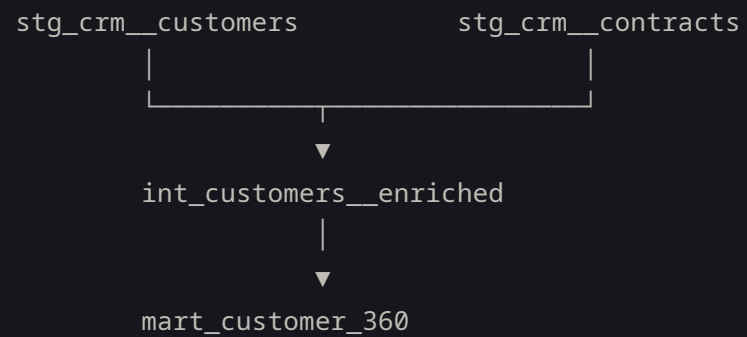
Typisk innhold:

- Joins mellom staging-modeller fra ulike kildesystemer
- Utpakking av JSON-felt som er for komplekse for staging
- Deduplisering av rader fra kilden
- Tekniske avledninger som gjelder entiteten generelt (f.eks. er kontrakten aktiv?)

```
int_customers__enriched.sql      ← CRM + kontrakter  
int_customers__activations.sql  
int_invoices__with_payments.sql
```

Navnekonvensjonen `int_<entitet>__<beskrivelse>` grupperer modeller etter entitet, ikke etter kilde.

Staging vs. intermediate vs. mart



En **staging**-modell: hva finnes i denne kildetabellen?

En **intermediate**-modell: hva vet vi om denne entiteten — uavhengig av hvem som spør?

En **mart**-modell: hva trenger dette spesifikke bruksområdet?

Hva skiller intermediate fra mart?

Intermediate inneholder logikk som er **universell for entiteten**:

- En kunde har én aktiv kontrakt (ikke konsumentspesifikt — det er alltid sant)
- En betaling er en duplikat hvis samme faktura er betalt to ganger (alltid sant)
- Kontraktsvarighet i dager (en egenskap ved kontrakten, ikke ved rapporten)

Mart inneholder logikk som er **spesifikk for konsumenten**:

- Livstidsverdi i euro (KPI for churn-teamet)
- Antall hendelser siste 90 dager (metrikk for nettverksdashboard)
- `is_at_risk` -flagg basert på betalingshistorikk (forretningsregel for én rapport)

*Det vanlige rådet er at «forretningslogikk hører hjemme i mart». Det stemmer for **konsumentspesifikk** logikk. Intermediate håndterer logikk som ville blitt duplisert på tvers av mange marts hvis den ikke ble løftet opp.*

JSON-utpakking — staging eller intermediate?

Dette er et designvalg, ikke et fasitsvar. To hensyn styrer beslutningen:

Pakk ut i staging når:

- JSON-feltet er flatt og enkelt
- Feltene brukes direkte som kolonner uten videre transformasjon
- Utpakkingen ikke avhenger av andre modeller
- Eksempel: `address` i `crm_customers`

Pakk ut i intermediate når:

- JSON-feltet er komplekst eller delvis null
- De utpakkede feltene må kombineres med data fra andre modeller for å gi mening
- Utpakket innhold er bare relevant i en spesifikk kontekst
- Eksempel: `preferences` og `device_info` — meningsfull først når vi vet hvem kunden er

Staging handler om kilden. Intermediate handler om entiteten.

JSON-utpakking i BigQuery

```
-- Flat utpakking med json_value()
json_value(preferences, '$.language')      as preferred_language,
json_value(preferences, '$.paperless')      as paperless_billing,
json_value(preferences, '$.marketing_opt_in') as marketing_opt_in,

-- Nested – device_info fra network_service_activations
json_value(device_info, '$.make')          as device_make,
json_value(device_info, '$.model')         as device_model,
json_value(device_info, '$.os')            as device_os,
json_value(device_info, '$.imei')          as device_imei,
```

`json_value()` returnerer alltid `STRING`. Cast eksplisitt hvis du trenger en annen type:

```
cast(json_value(preferences, '$.paperless') as bool) as paperless_billing
```

Tomme JSON-objekter `{}` gir `null` på alle `json_value()`-kall — håndter dette med `COALESCE` eller en test.

Tags i YAML

Tags lar deg gruppere modeller på tvers av mapper — nyttig for selektiv kjøring og dokumentasjon.

```
models:
  - name: int_customers__enriched
    config:
      materialized: table
      tags:
        - customer
        - daily
    columns:
      - name: customer_id
        description: Unik kundeidentifikator. Primærnøkkel.
      - name: preferred_language
        description: Foretrukket språk fra kundens preferanseprofil. Null hvis ikke satt.
```

Bruk tags til å kjøre et utvalg:

```
dbt run --select tag:customer
dbt build --select tag:daily
```


Kolonnenivå-beskrivelser i YAML

Beskrivelser på kolonnenivå fyller dokumentasjonssiden med innhold.

```
models:
  - name: int_customers__enriched
    description: >
      Én rad per kunde, beriket med aktiv kontraktinformasjon og
      utpakkede preferanser fra CRM.
    columns:
      - name: customer_id
        description: Primærnøkkel. Arvet fra stg_crm__customers.
      - name: contract_status
        description: >
          Status på kundens siste aktive kontrakt.
          Null for kunder uten noen kontrakt (f.eks. status pending).
      - name: monthly_fee_eur
        description: Månedlig totalbeløp på tvers av alle aktive kontrakter.
```

Skriv beskrivelsen slik at en ny kollega forstår kolonnen uten å se SQL-en.

Deduplisering

`billing_payments` inneholder dupliserte betalingsrader — samme faktura betalt to ganger i systemet.

En vanlig tilnærming med `ROW_NUMBER()`:

```
with payments_deduped as (  
  select  
    *,  
    row_number() over (  
      partition by invoice_ref  
      order by paid_on  
    ) as rn  
  from {{ ref('stg_billing__payments') }}  
)  
  
select * from payments_deduped  
where rn = 1
```

Spørsmål å tenke på:

- Hvilken rad beholder du — den første eller siste?
- Bør du filtrere i intermediate, eller eksponere `rn` til mart-laget?
- Burde det også finnes en **test** som fanger dette problemet?

Datasett: intermediate

Modellene vi skal bygge i denne blokken:

INTERMEDIATE-MODELL	HVA DEN GJØR
<code>int_customers__enriched</code>	Kobler <code>stg_crm__customers</code> med siste aktive kontrakt, pakker ut <code>preferences</code>
<code>int_customers__activations</code>	Kobler kunder med siste nettverksaktivering og pakker ut <code>device_info</code>
<code>int_billing__invoice_settlement</code>	Fakturaer beriket med betalingsstatus — faktura er primærentiteten, betaling er et oppslag
<code>int_network__incidents_enriched</code>	Kobler hendelser med kundeinformasjon

`crm_contracts` og `catalog_products` inneholder allerede historikk med `valid_from` / `valid_to` fra kildesystemet — staging eksponerer det som det er.

Blokk 4

Testing

Hva er en dbt-test?

En **test** i dbt er en sjekk som kjøres mot data i databasen og feiler hvis én eller flere rader ikke oppfyller kravet.

Tester kjøres med:

```
dbt test
dbt test --select stg__crm_customers # ett modell
dbt test --select tag:customer      # alle med tag
```

dbt skiller mellom to typer:

TYPE	BESKRIVELSE
Generiske	Makroer som konfigureres i YAML — gjenbrukbare
Singulære	SQL-filer i <code>data-tests/</code> — returnerer rader som feiler

Innebygde generiske tester

dbt har fire innebygde tester som dekker de vanligste behovene:

TEST	SJEKKER
<code>unique</code>	Ingen duplikater i kolonnen
<code>not_null</code>	Ingen null-verdier
<code>accepted_values</code>	Kun verdier fra en definert liste
<code>relationships</code>	Fremmednøkkel finnes i en annen modell

De konfigureres direkte i YAML — ingen SQL nødvendig.

Tester i YAML

```
models:
  - name: stg__crm_customers
    columns:
      - name: customer_id
        tests:
          - unique
          - not_null
      - name: status
        tests:
          - accepted_values:
              values: ['active', 'suspended', 'terminated']
      - name: contract_id
        tests:
          - relationships:
              to: ref('stg__crm_contracts')
              field: contract_id
```

Alle fire innebygde tester brukes i dette prosjektet — se `_staging.yml`.

Alvorlighetsgrad: **warn** vs. **error**

Ikke alle testfeil er like kritiske. dbt lar deg styre hva som skjer når en test feiler:

```
columns:
  - name: email
    tests:
      - not_null:
          severity: warn # logg advarsel, stopp ikke pipelinen
  - name: customer_id
    tests:
      - unique:
          severity: error # stopp pipelinen (standard)
```

- **error** (standard): dbt feiler — downstream-modeller kjøres ikke
- **warn**: dbt rapporterer problemet, men fortsetter

Bruk **warn** for kjente kildesystemfeil du dokumenterer men ikke kan fikse — f.eks. at 5 kunder mangler e-post.

store_failures

Når en test feiler ønsker du ofte å se hvilke rader som feiler, ikke bare at testen feiler.

```
columns:
  - name: customer_id
    tests:
      - relationships:
          to: ref('stg__crm_customers')
          field: customer_id
          store_failures: true
```

dbt materialiserer de feilende radene som en tabell i databasen — nyttig for feilsøking og dokumentasjon av kjente datakvalitetsproblemer.

Kan også settes globalt i `dbt_project.yml`:

```
tests:
  +store_failures: true
```

Singulære tester

En singulær test er en vanlig SQL-fil i `data-tests/` som returnerer rader som feiler.

```
-- data-tests/assert_invoice_amount_positive.sql
select *
from {{ ref('stg_billing_invoices') }}
where amount_eur < 0
      and status != 'credited'
```

dbt feiler testen hvis én eller flere rader returneres.

Passer for forretningsregler som ikke lar seg uttrykke med generiske tester — men kan bli mange og vanskelige å vedlikeholde i store prosjekter.

Testing av kjente feil

Kildedataene i dette prosjektet har 13 kjente problemer. Tester kan brukes til å dokumentere dem, ikke bare oppdage dem:

```
- name: customer_id
  tests:
    - unique:
        severity: warn
        # kjent: C001, C005, C010, C016 har to aktive kontrakter
```

To tilnærminger:

- **Ikke test det du vet feiler** — spar testen til kildesystemet er fikset
- **Test med `warn`** — dokumenterer problemet eksplisitt og varsler hvis det endrer seg

Hva er risikoen ved å ikke teste kjente feil i det hele tatt?

Datasett: tester i dette prosjektet

Viktige tester å ha på plass etter session 2:

MODELL	KOLONNE	TEST	MERK
stg__crm_customers	customer_id	unique , not_null	
stg__crm_contracts	customer_id	unique	forventes å feile — 4 kunder
stg__billing_invoices	customer_id	relationships	forventes å feile — 2 foreldreløse
stg__billing_payments	invoice_ref	unique	forventes å feile — duplikater
stg__crm_contracts	status	accepted_values	
int__customers_enriched	customer_id	unique , not_null	